

Algoritmos de Caminhos Mínimos em Grafos

Trabalho Prático 3 — Dijkstra, Bellman-Ford e Duan

Isabela M. Andrade Júlia A. da Rocha
Otávio de Oliveira Pedro Brassi Luccas

Departamento de Ciências da Computação
Universidade Federal de Alfenas

Maio de 2026

Problema: dado um grafo ponderado e um vértice de origem, encontrar a menor distância da origem até todos os demais vértices.

Aplicações: GPS, redes de computadores, transporte urbano, jogos digitais, redes logísticas, IA.

Algoritmos implementados:

- **Dijkstra** — abordagem gulosa clássica.
- **Bellman-Ford** — relaxamentos sucessivos de todas as arestas.
- **Duan** (2025) — estratégia híbrida com relaxamentos locais, redução de fronteira e refinamento hierárquico do processamento.

Objetivo: comparar empiricamente os três paradigmas em grafos ponderados, conexos e não orientados.

Dijkstra — ideia geral

- Calcula menores caminhos da origem a todos os vértices.
- Dois vetores auxiliares: `dist[]` (distâncias) e `visitado[]` (já finalizados).
- Inicialização: `dist[i] = INT_MAX/2`, `dist[origem] = 0`.
 - A divisão por 2 evita *overflow* nas somas do relaxamento.
- A cada iteração:
 - 1 escolhe vértice não visitado de menor `dist`;
 - 2 marca como visitado;
 - 3 *relaxa* suas arestas: se `dist[u] + peso(u,v) < dist[v]`, atualiza.

Complexidade

$\mathcal{O}(V^2)$ — matriz de adjacência + busca linear (**abordagem gulosa**).

Dijkstra — pseudocódigo

```
Algoritmo Dijkstra(matriz, n, origem)
  para i ← 0 até n-1 faça
    dist[i] ← INFINITO ; visitado[i] ← falso
  fim-para
  dist[origem] ← 0
  para iter ← 0 até n-1 faça
    menor ← INFINITO ; u ← -1
    para i ← 0 até n-1 faça
      se visitado[i] = falso e dist[i] < menor então
        menor ← dist[i] ; u ← i
    fim-se
  fim-para
  se u = -1 então retorna
  visitado[u] ← verdadeiro
  para cada vizinho v de u faça
    se dist[u] + peso(u,v) < dist[v] então
      dist[v] ← dist[u] + peso(u,v)
  fim-se
fim-para
```

- Resolve SSSP por **relaxamentos sucessivos** de todas as arestas.
- Diferente de Dijkstra: *não* seleciona o menor a cada passo.
- Permite **pesos negativos** (não explorado neste trabalho).
- Inicialização análoga ao Dijkstra (sem vetor visitado).
- **Otimização implementada:** parada antecipada quando nenhuma distância é alterada em uma rodada completa.

Complexidade

$\mathcal{O}(V^3)$ — três laços aninhados varrendo a matriz (**relaxamento global**).

Bellman-Ford — pseudocódigo

```
Algoritmo Bellman-Ford(matriz, n, origem)
  para i ← 0 até n-1 faça
    dist[i] ← INFINITO
  fim-para
  dist[origem] ← 0
  para k ← 1 até n-1 faça
    mudou ← falso
    para u ← 0 até n-1 faça
      para v ← 0 até n-1 faça
        se matriz[u][v] ≠ 0 então
          se dist[u] + matriz[u][v] < dist[v] então
            dist[v] ← dist[u] + matriz[u][v]
            mudou ← verdadeiro
          fim-se
        fim-se
      fim-para
    fim-para
    se mudou = falso então pare fim-se // parada antecipada
  fim-para
```

Primeiro algoritmo **determinístico** para SSSP a quebrar a barreira clássica de ordenação do Dijkstra. Substitui a ordenação global por uma **estrutura hierárquica** baseada em redução progressiva de fronteira, recursão controlada e relaxamentos locais em múltiplas escalas.

Parâmetros fundamentais: $k = \lfloor \log^{1/3}(n) \rfloor$ (iterações locais), $t = \lfloor \log^{2/3}(n) \rfloor$ (divisão hierárquica da fronteira).

Estruturas de dados:

- **Set** — conjuntos dinâmicos para fronteira S , alcançados W e pivôs P .
- **BlockQueue** — fila de blocos para gerenciar subproblemas recursivos.
- **MinHeap** — usado no Dijkstra limitado (caso base do BMSSP).

Complexidade

Teórica: $\mathcal{O}(m \log^{2/3} n)$. Na prática, o uso de matriz de adjacência impõe custo $\Theta(n^2)$ por varredura de vizinhos.

BMSSP — núcleo recursivo

Bounded Multi-Source Shortest Path

- **Entrada:** nível l , limite B , fronteira S .
- **Saída:** novo limite $B' \leq B$ e conjunto U de vértices completos ($d(v) < B'$).
- **Caso base** ($l = 0$): Dijkstra limitado com MinHeap.
- **Caso recursivo:** usa FindPivots, divide em blocos e recorre em $l - 1$.

FindPivots — redução de fronteira

Reduz S a um subconjunto de pivôs P .

- Executa k relaxamentos locais (Bellman-Ford restrito).
- Se $|W| > k \cdot |S| \Rightarrow$ fronteira densa, retorna $P = S$.
- Senão, constrói floresta sobre W e seleciona como pivôs os vértices de S com subárvore $\geq k$.

Ao final, um relaxamento completo garante a propagação correta das distâncias.

Duan — pseudocódigos

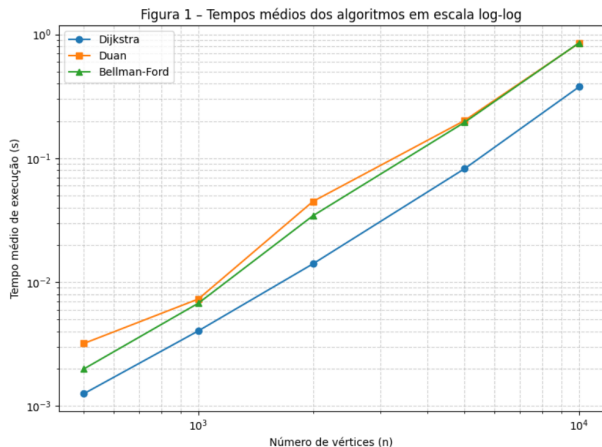
FindPivots — reduz fronteira S a um subconjunto de pivôs P :

```
Algoritmo FindPivots(B, S, k)
  W ← S
  repete k vezes
    Relaxar arestas de W (estilo Bellman-Ford, limitado por B)
    se |W| > k * |S| então retorne (S, W) fim-se // fronteira densa
  fim-repete
  Construir floresta F de caminhos minimos induzida por W
  P ← { u em S : tamanho_subarvore(u, F) ≥ k }
  retorne (P, W)
fim
```

BMSSP — recursão sobre blocos de fronteira:

```
Algoritmo BMSSP(l, B, S)
  se l = 0 então retorne Dijkstra_limitado(S, B) fim-se // MinHeap
  (P, W) ← FindPivots(B, S, k)
  D ← BlockQueue contendo P ; U ← vazio
  enquanto D nao vazio faça
    (Bi, Si) ← D.extract() ; (B'i, Ui) ← BMSSP(l-1, Bi, Si)
    U ← U uniao Ui ; Relaxar arestas de Ui
  fim-enquanto
  retorne (B', U)
```

Resultados — comparação geral



Tempo médio em função de n (escala log-log).

Configuração:

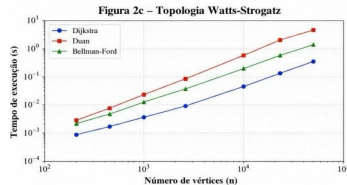
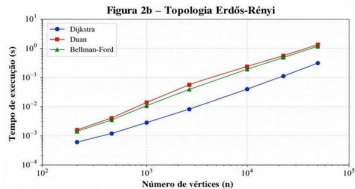
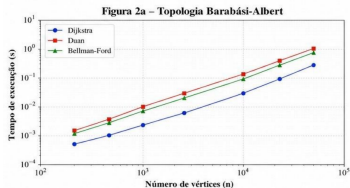
- 30 testes (3 topologias $\times$ 5 tamanhos $\times$ 2 sementes).
- $n = 500$ a 10 000 vértices.
- Tempo em segundos via `clock()`.

Resumo em $n = 10\,000$:

- **Dijkstra:** 0,38 s (1,00 $\times$, referência).
- **Duan:** 0,85 s (2,25 $\times$ mais lento).
- **Bellman-Ford:** 0,86 s (2,27 $\times$ mais lento).

Corretude: os três algoritmos retornaram o **mesmo custo ótimo** em todas as 30 instâncias.

Resultados — por topologia



Razão Duan/Dij em $n = 10\,000$:

- Barabási: $1,63\times$ mais lento.
- Erdős: $1,65\times$ mais lento.
- Watts: $4,12\times$ mais lento.

Duan é sensível à topologia: melhor em Barabási, pior em Watts-Strogatz (até $4,28\times$ mais lento em $n = 2000$).

Dijkstra

Mais rápido em todas as 30 execuções. A busca linear, embora teoricamente ineficiente para grafos esparsos, apresenta constante multiplicativa baixa quando combinada com matriz de adjacência.

Bellman-Ford

Confirmou seu comportamento de pior caso, sendo **consistentemente o mais lento ou o segundo mais lento**. A parada antecipada mitigou parcialmente o custo, mas não inverteu a ordem.

Duan

Aspectos positivos: **corretude total** (mesmos valores ótimos que Dijkstra e BF); competitivo em Barabási (até $1,24\times$ em $n = 1000$).

Aspectos negativos: **overhead significativo** da estrutura hierárquica (Set, BlockQueue, MinHeap); fortemente **sensível à topologia** (até $4,28\times$ mais lento em Watts).

Principais resultados:

- Dijkstra foi o algoritmo mais rápido em todas as 30 execuções (0,38 s em $n = 10\,000$, $2,25\times$ mais rápido que o Duan).
- Algoritmo de Duan está **completamente correto**, retornando os mesmos valores ótimos em todas as instâncias testadas.
- Bellman-Ford confirmou seu pior caso, sendo o mais lento na maioria das configurações.
- Os resultados empíricos confirmam as complexidades teóricas em ordem de magnitude.

Obrigado!

Perguntas?

Trabalho Prático 3 — Algoritmos de Caminhos Mínimos em Grafos
UNIFAL-MG — Maio de 2026